Zend

Relay Software Documentation

Architecture, API, packet handling, storage, and operational behavior

Strydr Silverberg

April 18, 2026

A complete technical guide to the Zend relay server, including request flow, blob lifecycle, runtime configuration, rate limiting, cleanup, and encrypted packet framing.

Contents

1 Overview **4**
- 1.1 What the Relay Does . . . 4
- 1.2 Design Goals . . . 4
- 1.3 System Architecture . . . 4
- 1.4 Source Files Covered . . . 5

2 Build-Time Defaults and Runtime Configuration **6**
- 2.1 Default Configuration . . . 6
- 2.2 Environment Variables . . . 6
- 2.3 Startup Sequence . . . 7

3 HTTP Server Structure **8**
- 3.1 Connection Handling . . . 8
- 3.2 Client IP Extraction . . . 8
- 3.3 Route Classification . . . 8
- 3.4 CORS Behavior . . . 9

4 Upload Flow **10**
- 4.1 Overview . . . 10
- 4.2 Upload Start . . . 10
  - 4.2.1 Endpoint . . . 10
  - 4.2.2 Behavior . . . 10
  - 4.2.3 Response Shape . . . 11
- 4.3 Upload Append . . . 11
  - 4.3.1 Endpoint . . . 11
  - 4.3.2 Expected Client Behavior . . . 11
  - 4.3.3 Validation Steps . . . 11
  - 4.3.4 Body Reading . . . 11
  - 4.3.5 Ordering Guarantee . . . 12
  - 4.3.6 Success Response . . . 12
- 4.4 Upload Finish . . . 12
  - 4.4.1 Endpoint . . . 12
  - 4.4.2 Behavior . . . 12
  - 4.4.3 Atomicity . . . 12
  - 4.4.4 Response Shape . . . 12

5 Download and Deletion Flow **13**
- 5.1 One-Time Download Semantics . . . 13
- 5.2 Download Endpoint . . . 13
  - 5.2.1 Behavior . . . 13
  - 5.2.2 Operational Implication . . . 13
- 5.3 Delete Endpoint . . . 13

5.3.1 Observed Logic . . . 14
5.3.2 Route Consistency . . . 14

6 On-Disk Storage Model **15**
6.1 File Types . . . 15
6.2 Path Construction . . . 15
6.3 Upload State File Format . . . 15
6.4 Metadata File Format . . . 16

7 Rate Limiting and Abuse Resistance **17**
7.1 Per-IP Accounting Model . . . 17
7.2 Window Reset . . . 17
7.3 HTTP Response on Rejection . . . 17
7.4 What This Protects Against . . . 17

8 Background Reaper and Expiration **19**
8.1 Purpose . . . 19
8.2 Completed Upload Reaping . . . 19
8.3 Incomplete Upload Reaping . . . 19
8.4 Cadence . . . 19

9 Blob Format and Packet Framing **20**
9.1 Why This Matters for the Relay . . . 20
9.2 Packet Types . . . 20
9.3 Frame Layout . . . 20
9.4 Chunking and Compression . . . 20
9.5 Metadata Packet Payload . . . 21
9.6 Integrity Packet Payload . . . 21
9.7 Done Packet Payload . . . 21

10 Encryption and Decryption Helpers **22**
10.1 AEAD Scheme . . . 22
10.2 Nonce Construction . . . 22
10.3 Stream Parsing on the Receive Side . . . 22

11 Logging and Observability **24**
11.1 Startup Logs . . . 24
11.2 Per-Request Logs . . . 24

12 End-to-End Blob Lifecycle **25**
12.1 Lifecycle Summary . . . 25
12.2 Normal Successful Flow . . . 25
12.3 Failure Paths . . . 25

13 Implementation Notes and Current Limitations **27**
13.1 Simple by Design . . . 27
13.2 Notable Limitations . . . 27
13.3 Strengths . . . 27

14 Conclusion **28**

Chapter 1

Overview

1.1 What the Relay Does

The Zend relay is a lightweight HTTP service written in Zig that accepts encrypted uploads, stores them temporarily on disk, and serves each uploaded blob back exactly once. The relay is intentionally narrow in scope: it does not perform encryption, key derivation, or plaintext inspection. Instead, it stores opaque ciphertext produced by the sender and deletes the stored blob after download or expiration.

At a high level, the relay is responsible for four jobs:

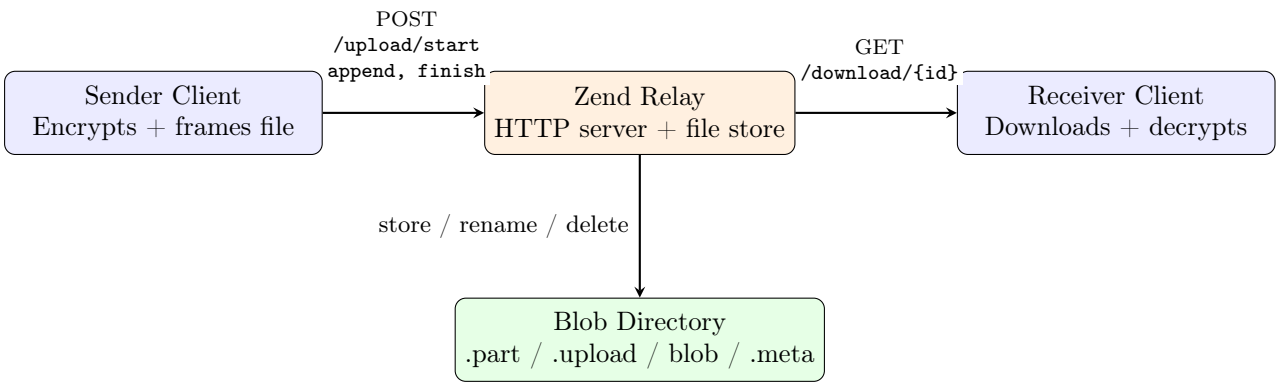
1. **Session creation** — issue an upload ID and upload token.
2. **Chunk intake** — append encrypted frame data to a temporary file in order.
3. **Finalization and serving** — atomically promote the completed upload and later return it to a downloader.
4. **Lifecycle management** — enforce size limits, rate limits, and time-to-live cleanup.

1.2 Design Goals

The implementation in the provided source files suggests a relay designed around the following goals:

- **Simplicity** — plain file-backed storage instead of a database.
- **Opaque transport** — the server handles ciphertext blobs without needing the file key.
- **Streaming-friendly uploads** — upload data arrives as ordered append requests rather than a single giant POST.
- **Short-lived storage** — completed and abandoned uploads are automatically reaped.
- **Small trusted computing base** — the relay mostly validates ordering, authorization, sizes, and file state.

1.3 System Architecture



1.4 Source Files Covered

This document is based on the relay-related source files currently available:

File	Role
`main.zig`	Listener setup, connection loop, route dispatch, CORS preflight, thread spawning
`runtime_config.zig`	Environment-variable based runtime configuration
`config.zig`	Default limits and timings
`http_helpers.zig`	Request body reading and plain-text responses
`upload.zig`	Upload start, append, and finish endpoints
`download.zig`	Download endpoint with consume-on-read deletion
`delete.zig`	Token-protected blob deletion endpoint
`storage.zig`	Path construction and file cleanup helpers
`rate_limit.zig`	Per-IP request accounting and route-aware quotas
`reaper.zig`	Expiration of complete and incomplete uploads
`ids.zig`	Random hexadecimal IDs and identifier validation
`blob_format.zig`	Encrypted frame format and stream parser helpers
`packet_types.zig`	Packet type and framing constants
`aead.zig`, `chacha20.zig`, `poly1305.zig`	Authenticated encryption implementation used by the protocol

Note: The relay itself primarily stores and serves opaque bytes. The packet-format and AEAD files are still included here because they define the blob structure that the relay transports and the clients later decrypt.

Chapter 2

Build-Time Defaults and Runtime Configuration

2.1 Default Configuration

The relay ships with a small set of hard-coded defaults in `config.zig`:

Name	Default	Meaning
`DEFAULT_PORT`	8080	HTTP listen port
`BLOB_DIR`	`blobs`	Directory used for blob and metadata files
`MAX_UPLOAD_SIZE`	512 MiB	Maximum completed blob size
`MAX_APPEND_BODY_SIZE`	1 MiB	Maximum body accepted by one append request
`TTL_SECONDS`	24 hours	Lifetime of a completed uploaded blob
`INCOMPLETE_TTL_SECONDS`	1 hour	Lifetime of abandoned partial uploads
`REAP_INTERVAL_NS`	60 seconds	Background cleanup interval
`RATE_LIMIT_WINDOW_SECONDS`	60	Rate-limit window size
`RATE_LIMIT_MAX_REQUESTS_PER_IP`	240	Total requests per IP per window
`RATE_LIMIT_MAX_UPLOAD_STARTS_PER_IP`	20	Upload session creations per window
`RATE_LIMIT_MAX_UPLOAD_APPENDS_PER_IP`	1200	Chunk append requests per window
`RATE_LIMIT_MAX_UPLOAD_FINISHES_PER_IP`	40	Upload finalizations per window
`RATE_LIMIT_MAX_DOWNLOADS_PER_IP`	120	Download requests per window

Table 2.1: Relay defaults from `config.zig`

2.2 Environment Variables

The relay resolves its runtime behavior from environment variables in `runtime_config.zig`. Each variable falls back to the default above when unset.

Environment Variable	Purpose
`ZEND_RELAY_HOST`	Bind host string
`ZEND_RELAY_PORT`	Bind port
`ZEND_BLOB_DIR`	Blob directory path
`ZEND_MAX_UPLOAD_BYTES`	Total allowed blob size
`ZEND_MAX_APPEND_BODY_BYTES`	Maximum append request size
`ZEND_TTL_SECONDS`	Completed blob lifetime
`ZEND_INCOMPLETE_TTL_SECONDS`	Partial upload lifetime
`ZEND_ALLOWED_ORIGINS`	CORS `Access-Control-Allow-Origin` value
`ZEND_RATE_LIMIT_WINDOW_SECONDS`	Rate-limit window length
`ZEND_RATE_LIMIT_MAX_REQUESTS_PER_IP`	Total request cap
`ZEND_RATE_LIMIT_MAX_UPLOAD_STARTS_PER_IP`	Start cap
`ZEND_RATE_LIMIT_MAX_UPLOAD_APPENDS_PER_IP`	Append cap
`ZEND_RATE_LIMIT_MAX_UPLOAD_FINISHES_PER_IP`	Finish cap
`ZEND_RATE_LIMIT_MAX_DOWNLOADS_PER_IP`	Download cap

Table 2.2: Runtime environment variables

2.3 Startup Sequence

When `main()` runs, it performs the following initialization sequence:

1. Create a general-purpose allocator.
2. Load runtime config from the environment.
3. Ensure the blob directory exists with `makePath()`.
4. Initialize the in-memory rate limiter.
5. Bind a TCP listener on `0.0.0.0:{port}`.
6. Spawn the background reaper thread.
7. Accept incoming TCP connections forever, creating a worker thread per connection.

At startup the relay logs the listen address, blob directory, upload limit, blob TTL, and rate-limit settings. This gives a concise operational snapshot in systemd logs.

Chapter 3

HTTP Server Structure

3.1 Connection Handling

Each accepted TCP connection is passed to `handleConnection()`, which constructs a Zig HTTP server around the connection stream and then processes requests in a loop until the client closes the connection.

Within that loop, the relay:

1. Parses the request head.
2. Normalizes the path by stripping any query string.
3. Logs the client IP, HTTP method, and request path.
4. Handles CORS `OPTIONS` requests immediately.
5. Applies per-IP rate limiting based on the classified route.
6. Dispatches to the upload, download, or delete handler.
7. Falls back to `404 Not found` for unknown paths.

3.2 Client IP Extraction

The relay derives an IP string from the accepted socket address using `clientIpString()`. The helper formats the address, then trims brackets for IPv6 and trims the final port component for IPv4-style `host:port` strings.

This string becomes the lookup key for rate limiting and is also included in request logs.

3.3 Route Classification

Before rate limiting, the relay maps each path to a route class:

Path Pattern	Route Kind
`/upload/start`	`upload_start`
`/upload/append/...`	`upload_append`
`/upload/finish/...`	`upload_finish`
`/download/...`	`download`
anything else	`other`

Table 3.1: Route classes used by the rate limiter

3.4 CORS Behavior

The relay handles CORS in two places:

- In `main.zig`, an `OPTIONS` request receives a `204 No Content` response with allow-origin, allow-methods, and allow-headers.
- Normal endpoint responses include `Access-Control-Allow-Origin` through `http_helpers.respondText()` or route-specific JSON/binary responses.

The allowed origin is not dynamically echoed from the request. Instead, it is emitted directly from `cfg.allowed_origins`. This keeps behavior simple, but it also means deployments need to set that value correctly for the web frontend.

Chapter 4

Upload Flow

4.1 Overview

Uploads are split into three HTTP actions:

1. `POST /upload/start`
2. `POST /upload/append/{id}?token=...&index=N`
3. `POST /upload/finish/{id}?token=...`

The relay stores upload state in two files during the transfer:

- `{id}.part` — partial ciphertext bytes
- `{id}.upload` — token, creation time, next expected chunk index, bytes written

This lets the relay remain stateless across requests except for the filesystem.

4.2 Upload Start

4.2.1 Endpoint

```
POST /upload/start
```

4.2.2 Behavior

`handleStart()` creates a new upload session as follows:

1. Reject any method other than `POST`.
2. Generate a random 16-character hexadecimal upload ID.
3. Generate a separate random 16-character hexadecimal upload token.
4. Create an empty temporary blob file `{id}.part`.
5. Create an upload state file `{id}.upload` initialized with:
   - token
   - creation timestamp
   - next expected append index = 0
   - bytes written = 0
6. Return JSON containing the ID and token.

4.2.3 Response Shape

```
{"id":"<upload-id>","token":"<upload-token>"}
```

4.3 Upload Append

4.3.1 Endpoint

```
POST /upload/append/{id}?token=<token>&index=<n>
```

4.3.2 Expected Client Behavior

Each append request contains one encrypted frame or a small bundle of already-framed ciphertext bytes. The relay does not parse those bytes; it only verifies ordering and total size before appending them to disk.

4.3.3 Validation Steps

`handleAppend()` performs the following checks in order:

1. Require method `POST`.
2. Extract the upload ID from the request path.
3. Extract the token and chunk index from the query string.
4. Parse the chunk index as `u32`.
5. Read the upload state file. If absent, return `404 Upload session not found`.
6. Compare the provided token to the stored token.
7. Require the provided index to match the stored `next_index` exactly.
8. Compute the maximum body size still allowed for this upload.
9. Read the request body with a hard cap using `http_helpers.readBody()`.
10. Verify that total bytes written plus body length does not exceed `max_upload_bytes`.
11. Open `{id}.part`, seek to the end, append the body, and sync.
12. Update `{id}.upload` with incremented `next_index` and new total size.

4.3.4 Body Reading

The helper `readBody()` reads the full body into memory using Zig's HTTP body reader, but it enforces a maximum length in two ways:

- It checks the declared `Content-Length`, when present.
- It checks the actual number of bytes read after streaming.

If the request exceeds the configured size bound, the append fails with `413 Payload Too Large`. In the current implementation, oversize appends also trigger cleanup of the upload's temporary files.

4.3.5 Ordering Guarantee

Append ordering is strictly enforced through the `index` query parameter and the `next_index` field in the state file. Any out-of-order or duplicated append results in a `409 Conflict` response.

This keeps the final on-disk blob identical to the client-side framed stream.

4.3.6 Success Response

```
{"ok":true}
```

4.4 Upload Finish

4.4.1 Endpoint

```
POST /upload/finish/{id}?token=<token>
```

4.4.2 Behavior

`handleFinish()` finalizes a completed upload:

1. Require method `POST`.
2. Extract upload ID and token.
3. Read the upload state file.
4. Verify the token.
5. Read the current size of `{id}.part`.
6. Re-check the size against `max_upload_bytes`.
7. Rename `{id}.part` to the final blob path `{id}`.
8. Write a metadata file `{id}.meta` containing:
   - the token
   - the completed-upload timestamp
9. Delete the `{id}.upload` state file.
10. Return JSON with the ID and token.

4.4.3 Atomicity

The rename from `.part` to the final blob path is the moment the upload becomes downloadable. This is a simple and effective atomic promotion step on a single filesystem.

4.4.4 Response Shape

```
{"id":"<upload-id>","token":"<upload-token>"}
```

Chapter 5

Download and Deletion Flow

5.1 One-Time Download Semantics

The relay's download path implements consume-on-read behavior. After a successful download response is sent, the relay deletes the blob and its metadata file.

> **Warning:** The current behavior is one-time download. Once a blob is served successfully, the relay immediately removes it from storage. This is a product choice, not just a storage optimization.

5.2 Download Endpoint

```
GET /download/{id}
```

5.2.1 Behavior

`handleDownload()` performs the following steps:

1. Require method `GET`.
2. Ensure the path starts with `/download/`.
3. Extract the blob ID from the suffix.
4. Read the full blob file into memory with an upper bound of `cfg.max_upload_bytes`.
5. Respond with `application/octet-stream` and the blob bytes.
6. Delete the blob file and its `.meta` file.

5.2.2 Operational Implication

Downloads are not streamed directly from disk to the socket in the current implementation. The entire ciphertext blob is first loaded into memory. That keeps the code small, but it also means peak memory use is tied to blob size.

5.3 Delete Endpoint

The source includes a dedicated delete handler meant to remove a stored blob when the caller presents the correct token.

```
DELETE or GET-like route handler intent:
/delete/{id}?token=<token>
```

5.3.1 Observed Logic

`handleDelete()`:

1. Splits the target into path and query components.
2. Extracts an ID from the path suffix.
3. Validates the ID format.
4. Extracts the `token=` query parameter.
5. Reads the first line of `{id}.meta` as the stored token.
6. Compares the presented token against the stored token.
7. Deletes the blob and metadata if the token matches.
8. Returns `204 No Content`.

5.3.2 Route Consistency

The delete path is now internally consistent across the relay:

- `main.zig` dispatches requests whose path begins with `/delete/`.
- `delete.zig` extracts the blob identifier using the same `/delete/` prefix.

As a result, a normal request of the form `/delete/{id}?token=...` reaches the handler, parses the identifier correctly, validates the token, and removes both the blob and its metadata when authorized.

Chapter 6

On-Disk Storage Model

6.1 File Types

The relay uses four file forms inside the blob directory:

Name	When it Exists	Purpose
`{id}.part`	During upload	Partial ciphertext bytes
`{id}.upload`	During upload	Upload session state
`{id}`	After finish, before download/expiry	Final ciphertext blob
`{id}.meta`	After finish, before download/expiry	Token and completion timestamp

Table 6.1: Blob directory file types

6.2 Path Construction

`storage.zig` centralizes path construction with helpers for:

- blob path
- metadata path
- temporary partial-blob path
- upload-state path

The same file also exposes cleanup helpers:

- `deleteBlob()` removes the final blob and its metadata.
- `deleteUploadTempFiles()` removes the partial blob and upload-state file.

6.3 Upload State File Format

The upload-state file stores one value per line:

```
<token>
<created_at_unix_timestamp>
<next_expected_index>
<bytes_written>
```

6.4 Metadata File Format

The metadata file stores:

```
<token>
<completion_unix_timestamp>
```

This is intentionally minimal. The relay does not track filename, content type, plaintext size, or key material in metadata.

Chapter 7

Rate Limiting and Abuse Resistance

7.1 Per-IP Accounting Model

The relay implements an in-memory rate limiter keyed by client IP string. Each entry tracks:

- window start time
- total requests
- upload starts
- upload appends
- upload finishes
- downloads

If a request would exceed either the total request cap or the route-specific cap, it is rejected.

7.2 Window Reset

Each IP entry is checked against the configured window duration. Once the window expires, all counters for that IP are reset and a new window begins at the current timestamp.

7.3 HTTP Response on Rejection

When the rate limiter denies a request, the relay replies with:

- status `429 Too Many Requests`
- `Content-Type: text/plain; charset=utf-8`
- `Retry-After: <seconds>`
- `Access-Control-Allow-Origin: <configured origin>`

7.4 What This Protects Against

The limiter is not a full abuse-prevention system, but it does help with:

- repeated session creation spam
- append flood attempts
- excessive downloads from a single IP
- generic high-volume probing

Note: Because the limiter is in-memory, counters reset when the relay restarts. That is reasonable for a small single-node relay, but persistent or distributed deployments would need a different backing store.

Chapter 8

Background Reaper and Expiration

8.1 Purpose

The relay includes a background cleanup thread so that storage cannot fill indefinitely with stale files.

8.2 Completed Upload Reaping

The reaper scans for `.meta` files. For each one, it:

1. Reads the completion timestamp from the second line.
2. Compares the current timestamp against `cfg.ttl_seconds`.
3. Deletes the corresponding final blob and metadata file if expired.

8.3 Incomplete Upload Reaping

The reaper also scans for `.upload` files. For each one, it:

1. Reads the creation timestamp from the second line.
2. Compares the current timestamp against `cfg.incomplete_ttl_seconds`.
3. Deletes the corresponding partial blob and upload-state file if abandoned long enough.

8.4 Cadence

The cleanup loop sleeps for `REAP_INTERVAL_NS`, which is currently 60 seconds, between passes.

> **Tip:** This is a practical design for a small relay. It avoids the complexity of per-upload timers and keeps the cleanup code easy to reason about.

Chapter 9

Blob Format and Packet Framing

9.1 Why This Matters for the Relay

Although the relay does not decrypt or inspect uploaded blobs, the blob format still matters because:

- upload clients append already-framed encrypted packets,
- download clients parse the stored bytes as a continuous stream, and
- the relay must preserve byte order exactly.

9.2 Packet Types

`packet_types.zig` defines four packet classes:

Constant	Hex	Purpose
`METADATA`	`0x03`	Filename and total plaintext size
`CHUNK`	`0x04`	One plaintext chunk, optionally compressed
`DONE`	`0x05`	End-of-stream marker with chunk count
`INTEGRITY`	`0x06`	Plaintext size and SHA-256 digest

Table 9.1: Protocol packet types

9.3 Frame Layout

Each encrypted frame has the following outer structure:

4 bytes	1 byte	variable	16 bytes
frame length	packet type	ciphertext	Poly1305 tag

The frame length is stored as a big-endian `u32` and covers the packet type byte, ciphertext, and authentication tag.

9.4 Chunking and Compression

Plaintext file content is divided into chunks of `64 KiB`. For each chunk, the client may Huffman-compress the plaintext. The chunk payload begins with:

- a `u32` little-endian chunk index
- a one-byte compression flag

If compression is not beneficial, the original chunk bytes are transmitted instead.

9.5 Metadata Packet Payload

The metadata plaintext contains:

- a `u16` little-endian filename length
- the filename bytes
- a `u64` little-endian total plaintext size

9.6 Integrity Packet Payload

The integrity plaintext contains:

- a `u64` little-endian total plaintext size
- a 32-byte SHA-256 digest of the complete plaintext

9.7 Done Packet Payload

The done plaintext contains a `u32` little-endian chunk count.

Chapter 10

Encryption and Decryption Helpers

10.1 AEAD Scheme

The provided protocol code uses ChaCha20-Poly1305 style authenticated encryption. The components are split across:

- `chacha20.zig` — stream cipher core
- `poly1305.zig` — authentication tag generation
- `aead.zig` — combined encrypt/decrypt interface

The test vectors in these files follow RFC 8439 style reference cases, and the implementation was validated against them successfully.

10.2 Nonce Construction

`blob_format.zig` builds a 12-byte nonce from:

- a 4-byte connection ID
- an 8-byte little-endian packet counter

In the current implementation, the connection ID in both encrypt and decrypt sessions is initialized to four zero bytes, while the counter starts at 1 and increments per packet.

> **Note:** That means uniqueness currently comes from the per-stream counter, not from a randomized connection identifier. If the same key were ever reused across multiple streams with the same starting nonce sequence, that would be dangerous for nonce-based AEAD. This document is describing the code as provided, not asserting that the current nonce strategy is the final intended production design.

10.3 Stream Parsing on the Receive Side

`DecryptSession.pushBytes()` supports incremental parsing of a blob stream:

1. Append new bytes to a pending buffer.
2. Wait until at least the 4-byte frame length is available.
3. Wait until the full frame is available.
4. Decrypt the ciphertext using the next nonce.
5. Process the packet based on its type.

6. Accumulate plaintext output from chunk packets.
7. Verify plaintext size and SHA-256 hash when the done packet arrives.

This parser is tolerant of partial frame arrival, which is important for streamed network reads.

Chapter 11

Logging and Observability

11.1 Startup Logs

The relay logs:

- listen port
- blob directory
- max upload size
- TTL values
- rate-limit settings

11.2 Per-Request Logs

Each request includes the client IP, method, and path. Additional route-specific logs exist for:

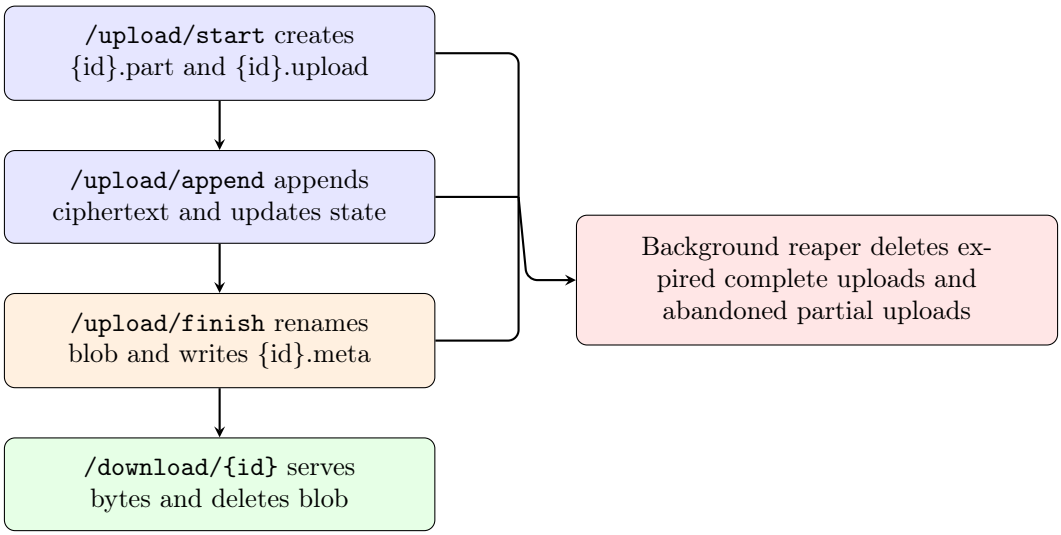
- upload start
- append begin / body length / file growth / commit
- append token or index errors
- finish promotion and size checks
- download start, size, serve, and consume
- reaper cleanup actions
- rate-limit rejections

This logging is detailed enough to reconstruct most upload and download failures from server logs alone.

Chapter 12

End-to-End Blob Lifecycle

12.1 Lifecycle Summary



12.2 Normal Successful Flow

1. Sender asks the relay for an upload session.
2. Relay returns ID and upload token.
3. Sender encrypts and frames the file locally.
4. Sender issues ordered append requests with the token and chunk index.
5. Relay appends bytes to the temporary file.
6. Sender finalizes the upload.
7. Relay promotes the temp file to a downloadable blob and writes metadata.
8. Receiver downloads the blob.
9. Relay deletes the blob immediately after serving it.
10. Receiver decrypts and verifies the file client-side.

12.3 Failure Paths

Common failure paths in the current implementation include:

- bad HTTP method

- missing or invalid token
- out-of-order append index
- oversize append body
- completed upload exceeding global size limit
- missing blob on download
- rate-limit rejection
- stale upload removed by the reaper before completion

Chapter 13

Implementation Notes and Current Limitations

13.1 Simple by Design

The relay is intentionally small, which makes it easy to audit and easy to run. That said, the current implementation also has some practical limitations worth recording.

13.2 Notable Limitations

1. **Download reads the full blob into memory.** For larger files, a fully streamed disk-to-socket download path would be more memory-efficient.
2. **Rate limiting is in-memory only.** Limits disappear on restart and do not coordinate across multiple relay instances.
3. **CORS allow-origin is static.** This is simple but requires careful deployment configuration.
4. **Metadata is minimal.** That keeps privacy exposure low, but it also means the relay has almost no introspection into stored uploads.
5. **Nonce construction should be reviewed carefully.** The protocol helper currently initializes connection IDs to zero in the provided code.

13.3 Strengths

Even with those caveats, the relay has several strong qualities:

- clear upload state machine
- exact append ordering enforcement
- clean atomic finalization step
- straightforward TTL-based garbage collection
- file-backed persistence without external dependencies
- good request-level logging for debugging

Chapter 14

Conclusion

The Zend relay is a compact, practical encrypted blob relay built around a filesystem-backed upload state machine. Its job is to accept opaque encrypted frames, preserve their order, expose the final ciphertext blob for download, and then clean up quickly. The server deliberately avoids deeper responsibilities such as plaintext processing or key management.

The relay already has the core properties needed for a working product prototype: bounded uploads, per-IP limits, upload resumption safety through index checking, background expiry, and simple token-based authorization for upload continuation and deletion. The main areas that still deserve attention are route consistency, download streaming efficiency, and a careful review of protocol nonce handling in the client/format layer.

End of document.